

Programa1.Normalizacion

September 24, 2026

Programa1.Normalizacion

September 23, 2026

Kevin Xavier Suasnavas Villarreal

[Link a mi GitHub](#)

1 Programa1.Normalizacion

```
[4]: # =====  
# Importación de librerías  
# =====  
  
# Importamos las herramientas de preprocesamiento de Scikit-Learn.  
# En este ejercicio utilizaremos MinMaxScaler para normalizar  
# los datos posteriormente.  
from sklearn import preprocessing  
  
# Importamos NumPy para trabajar con arreglos y matrices numéricas.  
import numpy as np
```

```
[5]: # =====  
# Carga de datos  
# =====  
  
# Creamos una matriz con los datos de entrenamiento.  
# Cada fila representa un registro y cada columna una característica.  
X_train = np.array([  
    [1., -4., 5.],  
    [5., 0., 2.],  
    [0., 6., -3.]  
)  
  
# Mostramos en pantalla los datos originales.  
print(X_train)
```

```
[[ 1. -4.  5.]
 [ 5.  0.  2.]
 [ 0.  6. -3.]]
```

```
[6]: # =====
# Limpieza de datos: normalización
# =====

# Creamos el objeto MinMaxScaler para normalizar los datos.
# Esta herramienta transforma los valores de cada característica
# a una escala comparable.
normalizer = preprocessing.MinMaxScaler()

# Ajustamos el normalizador utilizando los datos de entrenamiento
# y posteriormente transformamos dichos datos.
X_train_norm = normalizer.fit_transform(X_train)
```

```
[7]: # =====
# Salida de resultados
# =====

# Mostramos los datos después de aplicar la normalización.
print(X_train_norm)
```

```
[[0.2  0.  1.  ]
 [1.   0.4  0.625]
 [0.   1.   0.  ]]
```

```
[8]: # =====
# Ejemplo utilizando fit() y transform() por separado
# =====

# Creamos una copia de los datos originales para realizar
# nuevamente el proceso de normalización.
X_train_2 = np.copy(X_train)

# Ajustamos el normalizador utilizando los datos de entrenamiento.
# El método fit() aprende los valores mínimos y máximos
# necesarios para realizar la normalización.
f = normalizer.fit(X_train)

# Aplicamos el normalizador ya ajustado sobre los datos.
# El método transform() realiza la transformación utilizando
# los valores aprendidos anteriormente mediante fit().
X_train_norm_2 = f.transform(X_train)

# Comprobamos si el resultado obtenido es equivalente al
# resultado generado anteriormente con fit_transform().
```

```
print(np.array_equiv(X_train_norm, X_train_norm_2))
```

True

```
[9]: # =====  
# Creación del conjunto de datos de prueba  
# =====  
  
# Creamos un nuevo conjunto de datos que representa  
# información de prueba.  
X_test = np.array([  
    [1., -3., 2.],  
    [3., 0., 0.],  
    [0., 4., -1.]  
])
```

```
[10]: # =====  
# Normalización de los datos de prueba  
# =====  
  
# Aplicamos el mismo normalizador que fue ajustado  
# anteriormente con X_train.  
#  
# Importante:  
# No utilizamos fit() nuevamente porque el normalizador  
# ya aprendió los valores mínimo y máximo de X_train.  
# Para los datos de prueba solamente utilizamos transform().  
X_test_norm = normalizer.transform(X_test)  
  
# Mostramos los datos de prueba después de normalizarlos.  
print(X_test_norm)
```

```
[[0.2  0.1  0.625]  
 [0.6  0.4  0.375]  
 [0.   0.8  0.25 ]]
```

```
[ ]:
```